Verification Plan: 4 bit Alu

1.DUT Overview

The Dut is a 4bit ALU with no clk or reset signals, which performs arithmetic and logic operations on 2 input operands. Which operation is performed gets decided based on a selector input.

The RTL implementation is a black box, Verification is based solely on the given design specification

Signal	**Direction**	**Width**	**Description**
A	Input	4bit	Operand A
B	Input	4bit	Operand B
op	Input	3bit	Operation Selector
carry	Output	1bit	Carry(add/sub)or 0 for logic ops
Y	Output	4bit	ALU result

Opcode (op)	Operation	Description	Example (A=3, B=2)	Y	carry
000	ADD	Y = A + B	3 + 2	0101	Carry = 0
001	SUB	Y = A - B (two's complement)	3 - 2	0001	Carry = 0
010	AND	Bitwise AND	3 & 2	0010	0
011	OR	Bitwise OR	3 | 2	0011	0
100	XOR	Bitwise XOR	3 ^ 2	0001	0
101	NOT A	Bitwise negation of A	~3	1100	0
110	PASS A	Output A	3	0011	0
111	PASS B	Output B	2	0010	0

2.Verification Objectives

- Verify Carry is set correctly for Add/Sub and forced to 0 for rest
- Verify Y output for all 8 operations across all input combination
- All possible opcodes are defined so no verification for faulty opcode needed

3.Features to Verify

Feature	**Test**
ADD: Y= A + B (4bits)	All A,B combinations
ADD: carry = MSB of 5bit sum	All A,B combinations
SUB: Y= A-B mod 16	All A,B combinations
SUB: Carry bit	All A,B combinations
AND/OR/XOR/NOT: Correct Y	All A,B combinations
PASS A: Y=A	All A,B combinations
PASS B: Y=B	All A,B combinations
INVALID OPCODE: Y=0, Carry=0	None - all 8 opcodes possible are mapped

4.Testbench Architecture

1. Simple behavioral reference model implemented in SystemVerilog based on design specifications to predict expected value
2. Nested for loops iteration over opcodes and A, B operands feed both refrence model and DUT
3. Outputs get compared directly - Mismatches categorized in:
    a. Y mismatch,
    b. Carry mismatch
    c. Y & Carry mismatch
4. Per-opcode, reports Mismatches per category. If opcodes has mismatches displays all operands and corresponding Y and carry outputs categorized in Passing and Failing

5.Coverage Goals

Since the design is small enough we aim for a 100% coverage:

Input Space: 8 ops * 16 A * 16 B = 2048 test iterations

6.Tools

- Simulator: QuestaSim / ModelSim
- Language: SystemVerilog
- Waveform viewer: QuestaSim wave viewer (for the ADD waveform deliverable)
- VCD dump: for waveform capture

Results & Report

CODE

```
`timescale 1ns/1ps

module alu_compare_4bit (
   input  [2:0] op,
   input  [3:0] A,
   input  [3:0] B,
   output reg [3:0] Y,
   output reg       carry
);

   always @(*) begin
       case (op)
           3'b000: {carry, Y} = A + B;
           3'b001: {carry, Y} = A - B;
           3'b010: {carry, Y} = {1'b0, A & B};
           3'b011: {carry, Y} = {1'b0, A | B};
           3'b100: {carry, Y} = {1'b0, A ^ B};
           3'b101: {carry, Y} = {1'b0, ~A};
           3'b110: {carry, Y} = {1'b0, A};
           3'b111: {carry, Y} = {1'b0, B};
           default: {carry, Y} = 5'b00000;
       endcase
   end

endmodule


module alu_4bit_tb;

   reg  [2:0] _op;
   reg  [3:0] _A;
   reg  [3:0] _B;

   wire [3:0] _Y;
   wire       _carry;
```

```
wire [3:0] _Y_cmp;
wire       _carry_cmp;

integer op_i, a, b;
integer i;
integer errors;

integer y_mismatch      [0:7];
integer carry_mismatch  [0:7];
integer joined_mismatch [0:7];

reg [63:0] op_name;

alu_4bit dut (
    .A(_A),
    .B(_B),
    .op(_op),
    .Y(_Y),
    .carry(_carry)
);

alu_compare_4bit ref_model (
    .A(_A),
    .B(_B),
    .op(_op),
    .Y(_Y_cmp),
    .carry(_carry_cmp)
);

initial begin
    $dumpfile("alu_4bit.vcd");
    $dumpvars(0, alu_4bit_tb);

    errors = 0;

    for (i = 0; i < 8; i = i + 1) begin
        y_mismatch[i]      = 0;
        carry_mismatch[i]  = 0;
        joined_mismatch[i] = 0;
    end
```

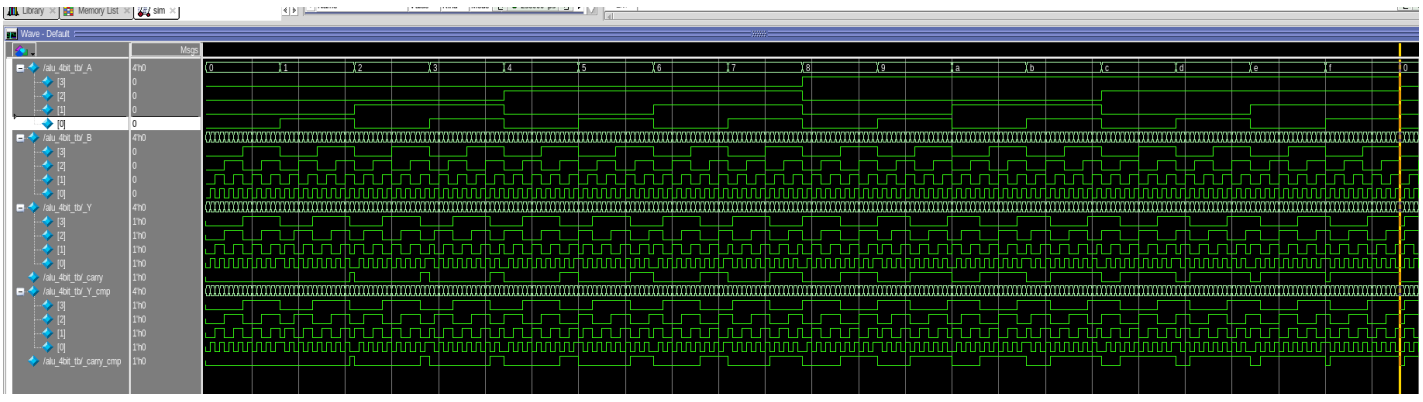
```
       for (op_i = 0; op_i < 8; op_i = op_i + 1) begin
           case (op_i)
               0: op_name = "ADD";
               1: op_name = "SUB";
               2: op_name = "AND";
               3: op_name = "OR";
               4: op_name = "XOR";
               5: op_name = "NOT A";
               6: op_name = "PASS A";
               7: op_name = "PASS B";
           endcase

           $display("\n-------- %s (%03b) --------", op_name, op_i[2:0]);

           // Pass 1: print all FAILs
           for (a = 0; a < 16; a = a + 1) begin
               for (b = 0; b < 16; b = b + 1) begin
                   _op = op_i[2:0];
                   _A  = a[3:0];
                   _B  = b[3:0];

                   #1;

                   if ((_Y !== _Y_cmp) || (_carry !== _carry_cmp)) begin
                       errors = errors + 1;

                       if ((_Y !== _Y_cmp) && (_carry !== _carry_cmp)) begin
                           joined_mismatch[op_i] = joined_mismatch[op_i] + 1;
                           $display("FAIL (Y+carry) A=%b B=%b -> DUT: Y=%b carry=%b
Ref: Y=%b carry=%b",
                                    _A, _B, _Y, _carry, _Y_cmp, _carry_cmp);
                       end
                       else if (_Y !== _Y_cmp) begin
                           y_mismatch[op_i] = y_mismatch[op_i] + 1;
                           $display("FAIL (Y only)  A=%b B=%b -> DUT: Y=%b
Ref: Y=%b",
                                    _A, _B, _Y, _Y_cmp);
                       end
                       else begin
                           carry_mismatch[op_i] = carry_mismatch[op_i] + 1;
                           $display("FAIL (carry)   A=%b B=%b -> DUT: carry=%b
Ref: carry=%b  Y=%b",
```

```
                                _A, _B, _carry, _carry_cmp, _Y);
                    end
                end
            end
        end

        // Pass 2: if this opcode had errors, print all PASSes
        if (y_mismatch[op_i] + carry_mismatch[op_i] + joined_mismatch[op_i] > 0)
begin
            $display("--- Correct combinations ---");
            for (a = 0; a < 16; a = a + 1) begin
                for (b = 0; b < 16; b = b + 1) begin
                    _op = op_i[2:0];
                    _A  = a[3:0];
                    _B  = b[3:0];

                    #1;

                    if ((_Y === _Y_cmp) && (_carry === _carry_cmp)) begin
                        $display("PASS           A=%b B=%b -> Y=%b carry=%b",
                                 _A, _B, _Y, _carry);
                    end
                end
            end
        end
    end

    if (errors == 0) begin
        $display("All combinations matched.");
    end
    else begin
        $display("\n================ FINAL SUMMARY ================\n");

        for (i = 0; i < 8; i = i + 1) begin
            case (i)
                0: op_name = "Add";
                1: op_name = "Sub";
                2: op_name = "AND";
                3: op_name = "OR";
                4: op_name = "XOR";
                5: op_name = "NOT A";
                6: op_name = "PASS A";
```

```
                    7: op_name = "PASS B";
                endcase

                $display("%s (%03b) :", op_name, i[2:0]);
                $display("Y mismatch      : %0d", y_mismatch[i]);
                $display("Carry mismatch  : %0d", carry_mismatch[i]);
                $display("Joined mismatch : %0d\n", joined_mismatch[i]);
            end

            $display("Total mismatches: %0d", errors);
        end

        $finish;
    end

endmodule
```

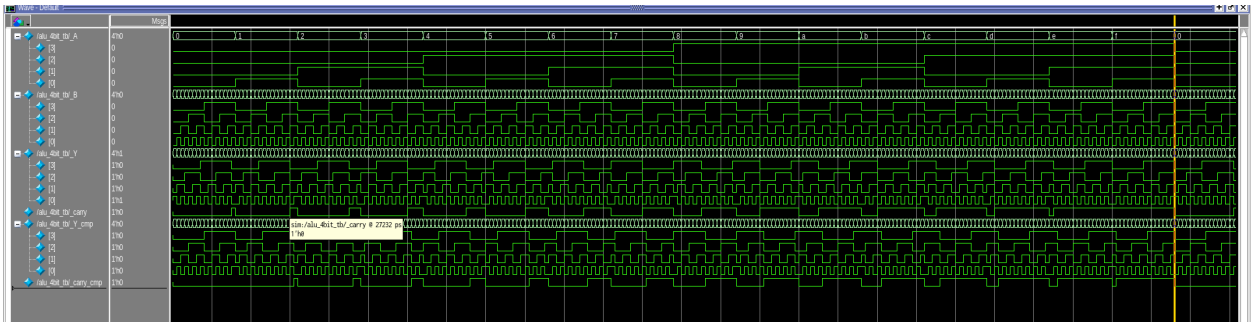
It's a total of 2048 tests - all possible combinations of inputs get tested. This gives me full coverage. As the module is very small such extensive testing does not run into any time issues.

Waveforms



Waveform of alu_4bit_v1. As visible the Pattern for Y and Y_cmp aswell as carry and carry_cmp are identical which matches the tb report of no mismatches between the DUT and

the reference model.



Waveform of alu_4bit_v5. Here the cmp and corresponding DUT signals do not match. This is consistent with the tb reporting that all adds mismatched with the reference outputs (failed the test) for this DUT version.

Results

Version	**Failed Ops**	**Reason for Failure**
v0	None	—
v1	None	—
v2	Sub	Carry not working for negative numbers (B > A)
v3	Or & And	**Or** behaves like And **And** behave like Or -> Opcode wrongly set
V4	None	—
V5	Xor, Add	Xor: 0 xor 0 results in 1 instead of 0 Add: +1 off the correct results

Side Notes

Time spent outside the Lab: 1.5 - 2h

AI usage :

Model: Claude code

Prompts (paraphrased) :

1. Clean Up the tb (clean up set in global claude context to not touch logic -> linting)
   -> added spacing, correct indents etc. Kinda exactly what a good linting program would do
2. Make the print statements prettier
   -> Adds Nicer formatted Headers, better description etc (e.g. see Final Summary print statement)
3. Add Print statements to also printing correct combinations if opcode has some failed combinations
   -> added logic for printing correct combinations